

Foundation Series · Board 2

ESP32 Basics

A powerful board with Wi-Fi and Bluetooth built in

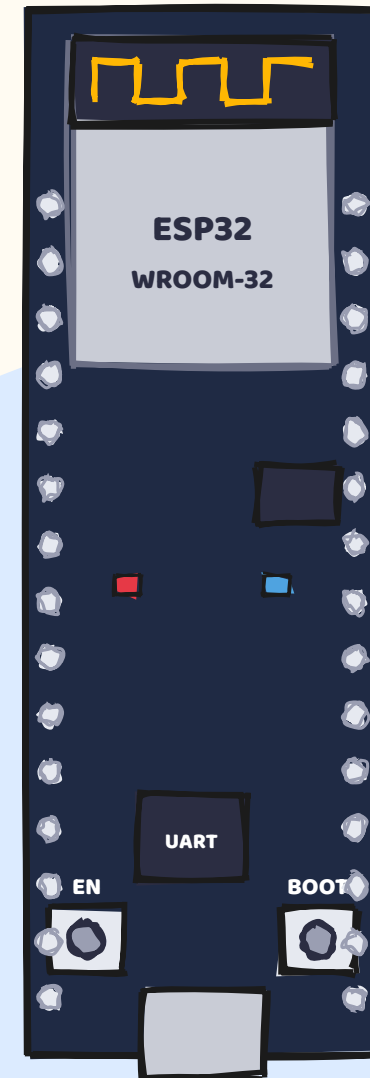
Dual-core 240 MHz

Wi-Fi + Bluetooth

3.3 V pins

Did you know?

Espressif, the company that makes the ESP32, has shipped over a billion chips for smart devices.





Think about it

How does a smart plug talk to your phone?



Smart bulbs

Change colour from a phone app



Smart doorbells

Send a message when someone rings



Plant monitors

Upload soil data to the cloud



Game controllers

Connect by Bluetooth



The Arduino Uno cannot connect to Wi-Fi on its own. Many smart devices use a chip with **wireless built in**, and the most popular one for makers is the **ESP32**.





Learning aims

Today I will learn to...



1  Describe what is **inside the ESP32** and how it differs from the Uno

2  Read the **DevKit pinout** and choose safe pins

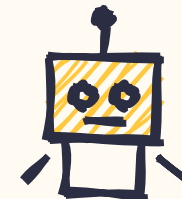
3  Use **3.3 V logic**, ADC, PWM, touch and DAC correctly

4  Connect to **Wi-Fi** and use **Bluetooth**

5  Set up the **Arduino IDE** and upload a first sketch

Keywords

SoC · module · GPIO · 3.3 V · ADC ·
PWM · Wi-Fi · BLE · strapping pin



Watch and think

Video time!



ESP32 For Dummies: Explained in 4 Minutes

Rolling Bob · about 4 minutes · youtu.be/6TjSI1cgijQ

Watch for...

- 1 What two **wireless** superpowers does the ESP32 have?
- 2 How is it different from the Arduino Uno?
- 3 Name one project you could build because it has Wi-Fi.

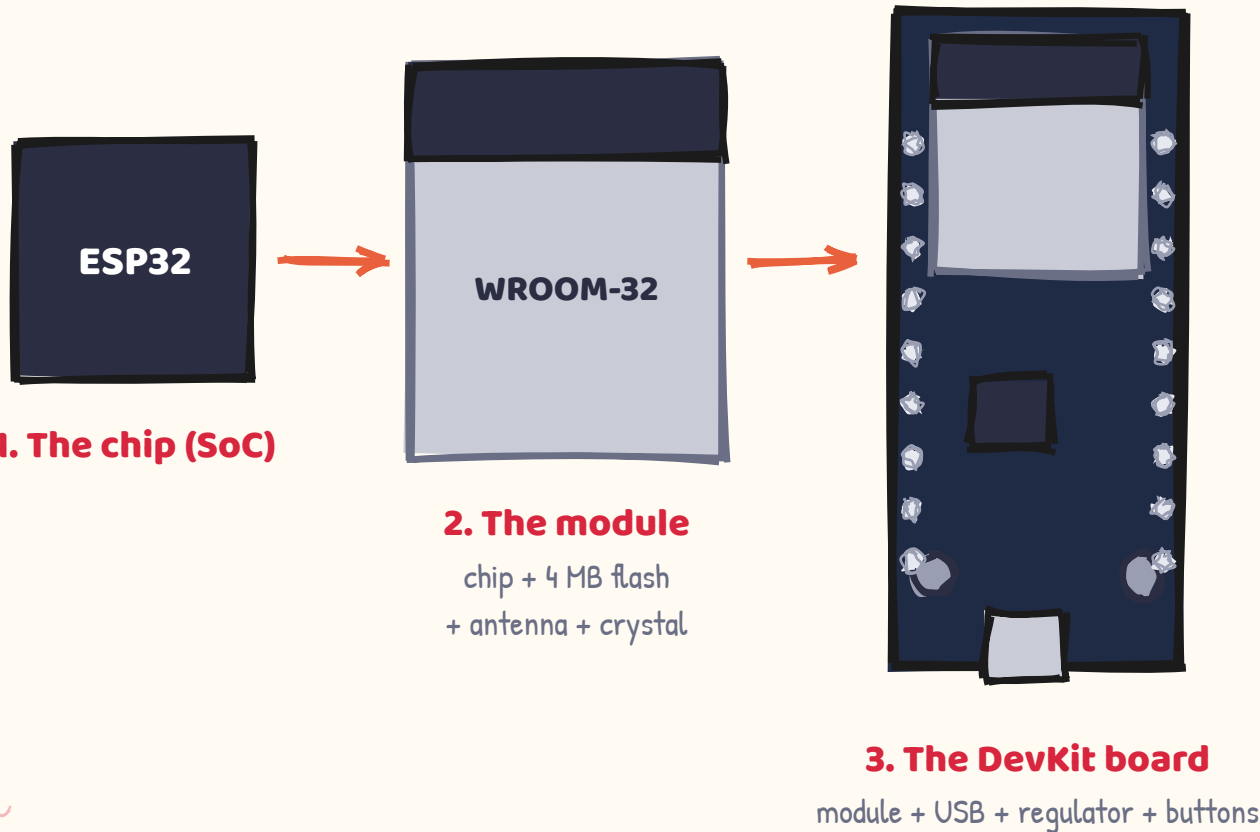


After the video: With your partner, list 2 things the ESP32 can do that our Arduino Uno cannot.



Three layers

Chip, module and development board



Who makes it?

Espressif Systems, a company from Shanghai, China, released the ESP32 in **2016**. It followed the popular ESP8266 Wi-Fi chip.



SoC = System on a Chip

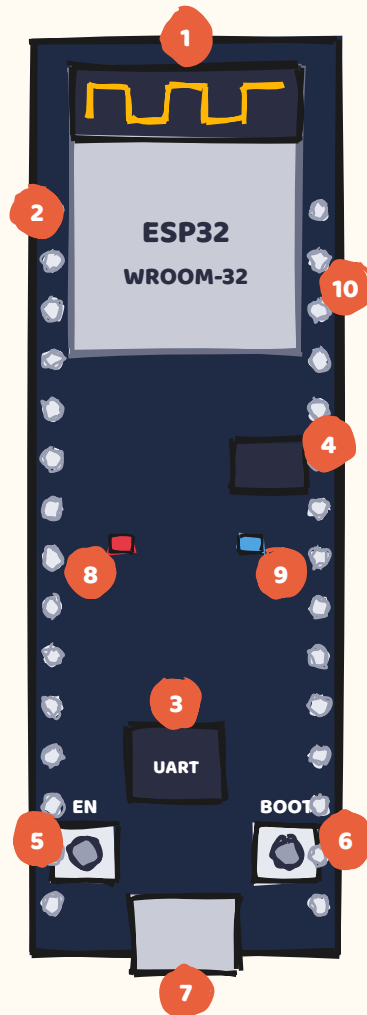
Processor, memory, Wi-Fi radio, Bluetooth radio and peripherals, all on one chip.





Take a tour

Parts of the ESP32 DevKit V1



- | | |
|--|--|
| <p>1 PCB antenna
Sends and receives radio</p> <p>3 USB-UART chip
CP2102 or CH340</p> <p>5 EN button
Reset / restart</p> <p>7 Micro-USB
Power + upload (USB-C on newer boards)</p> <p>9 Blue LED
GPIO2, like LED_BUILTIN</p> | <p>2 Pin headers
30 pins on DevKit V1</p> <p>4 3.3 V regulator
AMS1117: 5 V → 3.3 V</p> <p>6 BOOT button
GPIO0: hold to force upload mode</p> <p>8 Red LED
Power on</p> <p>10 Metal shield
WROOM-32 module inside</p> |
|--|--|



Check your board

DevKits come in 30-pin and 38-pin versions. Always read the labels printed on your board.





Inside the chip

ESP32 (WROOM-32) specifications



Processor	Xtensa LX6, 32-bit, dual-core , up to 240 MHz
Memory	520 KB SRAM , 448 KB ROM, 16 KB RTC memory
Flash	4 MB (on the WROOM-32 module)
Wi-Fi	802.11 b/g/n, 2.4 GHz only
Bluetooth	v4.2 Classic + Bluetooth Low Energy (BLE)
GPIO	34 on the chip (about 25 usable on a DevKit)
Analog	18 × 12-bit ADC, 2 × 8-bit DAC
Extras	10 touch pins, 16 PWM channels, ULP co-processor, hardware encryption
Logic level	3.3 V

Dual-core?

Two brains in one. In the Arduino core, Wi-Fi usually runs on **core 0** and your loop() on **core 1**, so the network does not freeze your code.



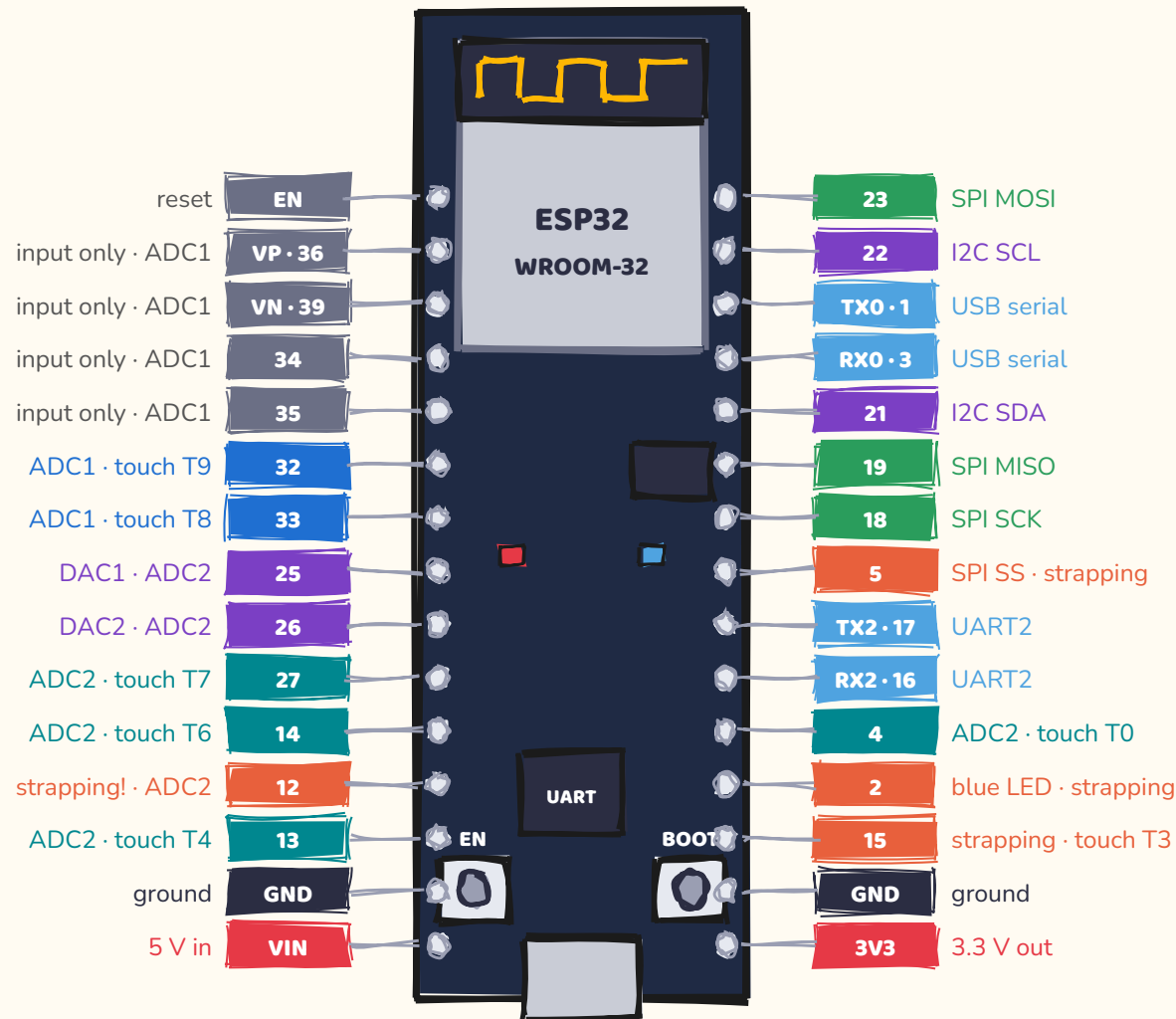
Speed check

240 MHz is 15 times the Uno's 16 MHz, with 260 times more RAM!



Know your pins

ESP32 DevKit V1 (30-pin) pin map



- Input only
- ADC1 (safe with Wi-Fi)
- ADC2 + touch
- DAC / I2C
- SPI
- UART
- Strapping pin
- Power

Numbers = GPIO
In code, use the GPIO number: 'D23' on the board is 23 in code.



Stay out of trouble

Four pin rules for the ESP32



3.3 V only!

Pins are **not 5 V tolerant**. A 5 V sensor signal can damage the chip. Use a level shifter or a voltage divider.



GPIO 34, 35, 36, 39

Input only, with no internal pull-up or pull-down.



GPIO 6 to 11

Connected to the flash memory. **Never use them.**
(Hidden on most DevKits.)



Strapping pins 0, 2, 5, 12, 15

Decide the boot mode. A wrong level at power-on can stop the board booting or uploading.



Beginner-safe pins: 4, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33



Same commands, new voltage

Digital input and output



HIGH	3.3 V (not 5 V)
LOW	0 V
pinMode	OUTPUT, INPUT, INPUT_PULLUP, INPUT_PULLDOWN
Current	Keep each pin under about 20 mA; about 40 mA is the absolute limit
LEDs	Still need a resistor (220-330 Ω)



Good news

pinMode(), digitalWrite(), digitalRead() and delay() work **exactly like on the Uno**. Most beginner sketches only need new pin numbers.



Bonus

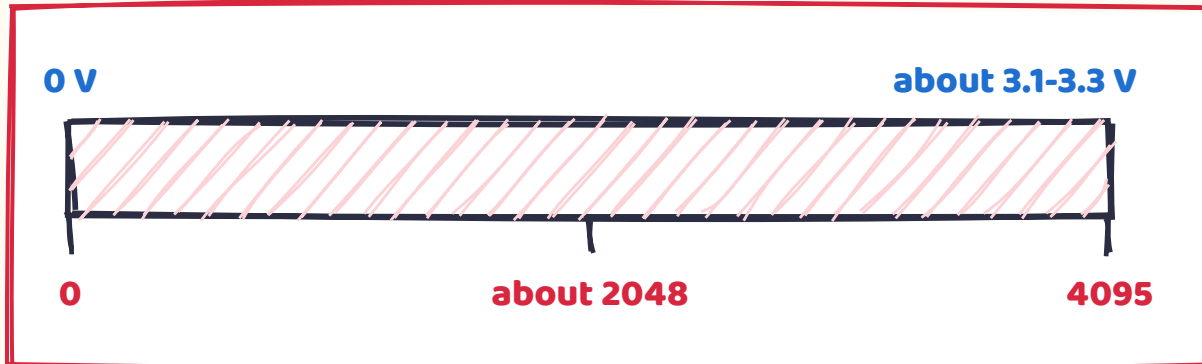
The ESP32 also has internal PULL-DOWN resistors. The Uno only has pull-ups.





12-bit analog input

Reading sensors with the ADC



```
int raw = analogRead(34);           // 0-4095
int mv  = analogReadMilliVolts(34); // calibrated mV
```

Pro tip

For sensor projects with Wi-Fi (like our plant monitor), put analog sensors on **ADC1 pins: 32, 33, 34, 35, 36, 39**.

Remember

Never put more than 3.3 V on an ADC pin.



Resolution	12-bit: 0 to 4095 (Uno: 0 to 1023)
ADC1	GPIO 32-39: works with Wi-Fi on
ADC2	GPIO 0, 2, 4, 12-15, 25-27: not usable while Wi-Fi is on
Accuracy	Not perfectly linear near 0 V and the top



LED control (LEDC)

PWM on the ESP32



Channels	16 independent PWM channels
Which pins?	Almost any output pin (not 34-39)
Frequency	You choose: e.g. 5 kHz for LEDs, 50 Hz for servos
Resolution	You choose: e.g. 8-bit (0-255) up to 16-bit

```
ledcAttach(2, 5000, 8); // pin, 5 kHz, 8-bit  
ledcWrite(2, 128);      // about 50% brightness
```

Uno vs ESP32

Uno: 6 fixed PWM pins, fixed frequency.
ESP32: pick the pin, frequency and resolution.
analogWrite() also works in the newer ESP32 core.

Version note

These commands are for ESP32 Arduino core 3.x. Old tutorials use `ledcSetup()` and `ledcAttachPin()`.



Two special features

Touch sensing and true analog output



Capacitive touch: 10 pins

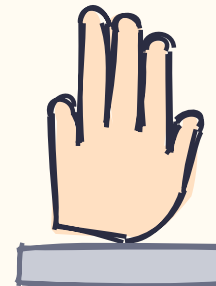
Your finger changes the pin's capacitance. On the ESP32, **touchRead()** gives a **lower** number when touched. No button needed!

Touch pins: GPIO 0, 2, 4, 12, 13, 14, 15, 27, 32, 33

DAC: 2 real analog outputs

GPIO 25 and 26 output a real voltage, 8-bit (0-255 → 0 to about 3.3 V). Used for simple audio and signals. The Uno has no DAC.

```
int t = touchRead(T0); // T0 = GPIO4
if (t < 30) { /* touched! */ }
dacWrite(25, 128);      // about 1.65 V on GPIO25
```



foil pad → GPIO4

Try it

Tape foil to a jumper wire on GPIO4 and build a touch lamp.



Talking to other chips

Communication on the ESP32



Bus	How many	Default pins
UART	3 ports	UART0 = GPIO1/3 (USB), UART2 = GPIO17/16
I2C	2 buses	Default SDA 21, SCL 22 (can be moved)
SPI	4 (2 free to use)	VSPI: MOSI 23, MISO 19, SCK 18, SS 5
I2S	2	Digital audio: microphones, speakers
CAN (TWAI)	1	Car and industrial networks

```
Wire.begin(21, 22); // I2C: SDA, SCL
```

OLED screens, MPU6050 and most I2C modules work on 3.3 V, so they connect to the ESP32 directly.



Pin freedom

The ESP32 has a pin matrix, so most buses can be **moved to other pins** in code.



Uno vs ESP32

Uno: 1 UART shared with USB. ESP32: spare UARTs for GPS or other boards.





Built-in wireless

Connecting to Wi-Fi



```
#include <WiFi.h>

const char* ssid = "SchoolLab";
const char* pass = "your-password";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);           // join a router
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(WiFi.localIP()); // e.g. 192.168.1.23
}

void loop() { }
```

Station

joins a router

Access Point

makes its own network

2.4 GHz only!

The ESP32 cannot see 5 GHz networks. If your network is missing, check the router band. Networks with a login web page (like some school Wi-Fi) do not work easily.

Once connected, the ESP32 can host a web page, send data to the cloud or talk to a phone app.



Built-in wireless

Bluetooth Classic and BLE

```
#include "BluetoothSerial.h"
BluetoothSerial BT;

void setup() {
  BT.begin("ESP32-Robot");    // name seen by phones
  pinMode(2, OUTPUT);
}

void loop() {
  if (BT.available()) {
    char c = BT.read();
    if (c == '1') digitalWrite(2, HIGH);
    if (c == '0') digitalWrite(2, LOW);
  }
}
```

Type	Best for	Examples
Classic	Continuous data, like a wireless cable	Serial Bluetooth apps, robot cars
BLE	Low energy, short messages	Fitness bands, beacons, sensors

Test it

Pair an Android phone with "ESP32-Robot", open a **Serial Bluetooth Terminal** app and send 1 or 0.

BluetoothSerial works on the original ESP32 only. iPhones need BLE instead of Classic.



Power smart

Powering the ESP32 and saving energy



USB	5 V → regulator → 3.3 V
VIN pin	5 V in (same as USB)
3V3 pin	3.3 V out for sensors, or 3.3 V in
Wi-Fi bursts	Up to about 240 mA

Mode	Chip current
Active + Wi-Fi	about 95-240 mA
Modem sleep	about 20-68 mA
Light sleep	about 0.8 mA
Deep sleep	about 10 μ A
Hibernation	about 5 μ A

Brownout!

"Brownout detector was triggered" means the power dipped when Wi-Fi started. Use a **good USB cable** and a supply of at least **500 mA**.

Why it matters

Deep sleep lets a battery weather station wake, send data and sleep again for months. The DevKit's LED and regulator use more than the chip.





Get ready to code

Setting up the Arduino IDE for ESP32



- 1 File → Preferences → Additional boards manager URLs, add:
https://espressif.github.io/arduino-esp32/package_esp32_index.json
- 2 Tools → Board → Boards Manager: install
esp32 by Espressif Systems
- 3 Tools → Board → esp32 → choose
ESP32 Dev Module
- 4 Install the USB driver if no port appears
CP210x or CH340 (check the chip on your board)
- 5 Tools → Port → choose the new COM port, then Upload

Stuck on 'Connecting...'?

Hold the **BOOT** button when the dots appear, release when uploading starts.

Charge-only cables

Some USB cables only carry power. If no port appears, try another cable first!





Your first ESP32 sketch

Blink the blue LED

```
const int LED = 2;           // blue LED on most DevKits

void setup() {
  Serial.begin(115200);       // ESP32 speaks faster
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
  Serial.println("LED on");
  delay(500);
  digitalWrite(LED, LOW);
  delay(500);
}
```

What is different?

The LED is on **GPIO 2**, not 13.
Serial Monitor must be set to **115200** baud.
After upload, press **EN** if nothing happens.

Try it

Change the LED to GPIO 4 with a resistor and your own LED. Then make it fade with `ledcWrite()`.





Side by side

Arduino Uno vs ESP32



	Arduino Uno R3	ESP32 DevKit
Processor	8-bit AVR, 1 core, 16 MHz	32-bit Xtensa, 2 cores, 240 MHz
Flash / RAM	32 KB / 2 KB	4 MB / 520 KB
Logic level	5 V	3.3 V
Analog in	6 × 10-bit	18 × 12-bit
Analog out	None (PWM only)	2 DAC + 16 PWM channels
Wireless	None	Wi-Fi + Bluetooth
Touch pins	None	10
Best for	Learning, 5 V sensors and shields	IoT, apps, dashboards, fast projects

Which one?

Start with the Uno to learn. Move to the ESP32 when your project needs the internet or more speed.





Meet the family

The ESP32 family



ESP32-S2

Single core, Wi-Fi only, native USB



ESP32-S3

Dual core, Wi-Fi + BLE 5, AI instructions, native USB



ESP32-C3

RISC-V single core, Wi-Fi + BLE 5, low cost



ESP32-C6

Wi-Fi 6, BLE 5, Zigbee and Thread



ESP32-CAM

ESP32 + OV2640 camera + microSD slot

In our lab

We use the ESP32 DevKit for IoT projects and the ESP32-CAM for camera and AI projects.



All of them use the **same Espressif board package** in the Arduino IDE. Just pick the right board in Tools → Board.



Show what you know

Recap and quick quiz

Remember

- ☒ Dual-core 240 MHz, 520 KB SRAM, 4 MB flash
- ☒ Wi-Fi (2.4 GHz) and Bluetooth built in
- ☒ 3.3 V pins: never connect 5 V signals
- ☒ Use ADC1 pins when Wi-Fi is on
- ☒ Avoid GPIO 6-11 and be careful with strapping pins

Quick quiz

1. Why can't GPIO 34 drive an LED?
2. Which analog pins still work with Wi-Fi on?
3. What does the BOOT button do?
4. Why won't the ESP32 join a 5 GHz network?

Challenge

Make a touch pad on GPIO4 that switches the blue LED, then control it by Bluetooth too.

